

PYTHON • PRACTICE SET

File Handling Problem Set

Reading, transforming & editing files — with modules & packages

2 x Text files

2 x CSV files

Read → Solve → Write

Standard library

PROBLEM 01 • TEXT

Server Log Analyzer

Parse a log file, count severities, extract errors, write a summary.

PROBLEM 02 • TEXT

Document Redaction Tool

Mask sensitive terms with regex and emit a release-safe copy.

PROBLEM 03 • CSV

Sales Report Generator

Aggregate revenue by category and rank the top-selling product.

PROBLEM 04 • CSV

Student Result Processor

Compute totals, averages & grades, then write an enriched CSV.

THE WORKFLOW — APPLY IT TO EVERY PROBLEM

1 Read the file

`open() / csv / with`



2 Solve with a module

`collections / re / csv`



3 Print or write

`console / new .txt / .csv`

Each task: read the file → solve with the right module → print or write the result.

4 PROBLEMS

1

Server Log Analyzer

TEXT FILE

SCENARIO

You are a junior DevOps engineer. Your application writes runtime events to a log file **app.log**. Each line follows the format `TIMESTAMP LEVEL MESSAGE`, where `LEVEL` is one of `INFO`, `WARNING`, `ERROR` or `DEBUG`. The on-call team wants a quick daily summary of how many events of each severity occurred, plus a list of every error that was logged.

SAMPLE INPUT

```
2026-01-15 09:12:04 INFO User login successful for user_id=204
2026-01-15 09:13:22 WARNING Disk usage at 82%
2026-01-15 09:15:41 ERROR Database connection timeout
2026-01-15 09:16:03 INFO Cache refreshed
2026-01-15 09:18:55 ERROR Null pointer in payment module
```

REQUIREMENTS

1. Create a module **log_utils.py** with two functions: `parse_line(line)` returning a `(level, message)` tuple, and `read_logs(path)` returning a list of parsed entries.
2. In **main.py**, import from `log_utils` and read **app.log**.
3. Count entries per severity level using `collections.Counter`.
4. Extract all `ERROR` messages into a list.
5. Print the summary to the console **and** write it to a new file **log_summary.txt**.

MODULES & PACKAGES

`collections.Counter``log_utils (custom)``open() / write()`

EXPECTED OUTPUT

```
=== Log Summary ===
INFO      : 2
WARNING   : 1
ERROR     : 2
DEBUG     : 0

Errors found:
- Database connection timeout
- Null pointer in payment module
```

2

Document Redaction Tool

TEXT FILE

SCENARIO

A company must publish internal reports externally, but sensitive terms such as project code names and client names have to be hidden first. Build a redaction tool that reads a report, masks all sensitive terms, and produces a safe version for release.

SAMPLE INPUT

report.txt

DATA

```
The Titan project launched with client Acme Corp last quarter.  
Titan exceeded targets, and Acme Corp renewed the contract.  
Our next initiative, Project Titan 2, begins in March.
```

REQUIREMENTS

1. Create a module **redaction_config.py** that defines `SENSITIVE = ["Titan", "Acme Corp"]`.
2. In **main.py**, import that list and read **report.txt**.
3. Replace every occurrence of each term (case-insensitive) with `[REDACTED]` using the `re` module.
4. Count how many replacements were made per term.
5. Write the cleaned text to **report_redacted.txt** and print a redaction count report to the console.

MODULES & PACKAGES

`re`

`redaction_config` (custom)

`open()` / `write()`

EXPECTED OUTPUT

report_redacted.txt + console

OUTPUT

```
-- report_redacted.txt --  
The [REDACTED] project launched with client [REDACTED] last quarter.  
[REDACTED] exceeded targets, and [REDACTED] renewed the contract.  
Our next initiative, Project [REDACTED] 2, begins in March.  
  
-- console --  
Redaction complete.  
Titan      -> 3 occurrences redacted  
Acme Corp  -> 2 occurrences redacted  
Output saved to report_redacted.txt
```

3

Sales Report Generator

CSV FILE

SCENARIO

A retail store exports its daily transactions to **sales.csv**. Management wants an automated report showing revenue by category, the single best-selling product by revenue, the total revenue, and the average value per transaction.

SAMPLE INPUT

```
sales.csv DATA
date,product,category,quantity,unit_price
2026-01-10,Wireless Mouse,Electronics,5,650
2026-01-10,Notebook,Stationery,20,45
2026-01-10,USB Cable,Electronics,10,120
2026-01-11,Pen Pack,Stationery,15,90
2026-01-11,Keyboard,Electronics,3,1200
```

REQUIREMENTS

1. Read **sales.csv** using the csv module (csv.DictReader).
2. For each row compute revenue as $\text{quantity} \times \text{unit_price}$.
3. Aggregate total revenue per category.
4. Identify the single product with the highest revenue.
5. Use the statistics module to compute the average revenue per transaction.
6. Print a clean, formatted report to the console.

MODULES & PACKAGES

csv.DictReader

statistics.mean

collections (optional)

EXPECTED OUTPUT

```
console OUTPUT
=== Sales Report ===
Revenue by Category:
  Electronics : 8050.00
  Stationery  : 2250.00

Top Product   : Keyboard (3600.00)
Total Revenue : 10300.00
Avg / Txn     : 2060.00
```

4

Student Result Processor

CSV FILE

SCENARIO

A college stores exam marks in **students.csv**. The examination office needs a processed file that adds each student's total, average and grade, plus a quick console summary showing the class topper and the pass / fail count.

SAMPLE INPUT

```
students.csv DATA
roll_no,name,maths,physics,chemistry
101,Aarav,78,85,90
102,Diya,30,25,35
103,Kabir,92,88,95
104,Meera,60,58,65
```

REQUIREMENTS

1. Read **students.csv** using `csv.DictReader`.
2. For each student compute the total and average of the three subjects.
3. Assign a grade from the average: A ≥ 90 , B 75–89, C 60–74, D 40–59, F < 40 . (Extension: put this in a module **grading.py**.)
4. Write **students_result.csv** using `csv.DictWriter` — all original columns plus total, average, grade.
5. Print the class topper and the number of students who passed ($\text{avg} \geq 40$) versus failed.

MODULES & PACKAGES

`csv.DictReader`

`csv.DictWriter`

`grading` (optional)

EXPECTED OUTPUT

```
students_result.csv + console OUTPUT
-- students_result.csv --
roll_no,name,maths,physics,chemistry,total,average,grade
101,Aarav,78,85,90,253,84.33,B
102,Diya,30,25,35,90,30.0,F
103,Kabir,92,88,95,275,91.67,A
104,Meera,60,58,65,183,61.0,C

-- console --
Processed 4 students -> students_result.csv
Class Topper : Kabir (avg 91.67)
Passed : 3 | Failed : 1
```